

Module 3 - Logic and Loops

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)
 - [Escape Sequences](#)
 - [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)
- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)
 - [Boolean Expressions](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Ternary Operator \(Conditional Operator\)](#)
 - [Membership Operators](#)
 - [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)
 - [Loop Control Statements](#)
 - [Infinite Loops](#)
 - [Nesting Loops](#)
 - [Common Loop Patterns](#)
 - [Choosing the "Right" Loop](#)
 - [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

Working With Strings

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)
 - [Escape Sequences](#)
 - [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)
- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)

- [Boolean Expressions](#)
- [Comparison Operators](#)
- [Logical Operators](#)
- [Ternary Operator \(Conditional Operator\)](#)
- [Membership Operators](#)
- [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)
 - [Loop Control Statements](#)
 - [Infinite Loops](#)
 - [Nesting Loops](#)
 - [Common Loop Patterns](#)
 - [Choosing the "Right" Loop](#)
 - [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

String Methods

The String data type is encapsulated with a great deal of functionality which can be quite useful to the software developer. This section will briefly describe some of the most popular string methods. More information can be found [here](#).

`capitalize()`: Converts the first character of a string to uppercase.

```
message = "hello, world!"
message = original_string.capitalize()

print(message)
```

Output

```
Hello, world!
```

`center()`: Center-aligns a string within a specified width, padding it with a specified character (or a space by default).

```
message = "Python"
width = 10
fill_character = "-"

message = message.center(width, fill_character)

print(message)
```

Output

```
--Python--
```

`startswith()`: Returns true if a string starts with the specified prefix.

```
message = "Hello, World"
boolean_variable = message.startsswith("World")
print(boolean_variable)
boolean_variable = message.startswith("Hello")
print(boolean_variable)
```

Output

```
False
True
```

`endswith()`: Returns true if a string ends with the specified suffix.

```
message = "Hello, World"
boolean_variable = message.endswith("World")
print(boolean_variable)
boolean_variable = message.endswith("Hello")
print(boolean_variable)
```

Output

```
True
False
```

`len()`: Returns the length of a string.

```
message = "Hello, World!"
length = len(message)
print(length)
```

Output

```
13
```

`upper()`: Converts the string to uppercase characters.

```
message = "Hello, World!"
message = message.upper()
print(message)
```

Output

```
HELLO, WORLD!
```

`lower()`: Converts the string to lowercase characters.

```
message = "Hello, World!"  
message = message.lower()  
  
print(message)
```

Output

```
hello, world!
```

`replace()`: Replaces all occurrences of a substring within another substring.

```
message = "Hello, World! Hello, Python!"  
new_string = message.replace("Hello", "Hi")  
  
print(message)
```

Output

```
Hi, World! Hi, Python!
```

`str()`: Converts other data types to string.

```
age = 42  
age = str(integer_value)  
  
print(age)
```

Output

```
42
```

`split()`: Splits the string into a list of substrings based on a specified delimiter (default delimiter is a comma).

```
message = "apple,banana,orange"  
fruits_list = message.split(',')  
  
print(fruits_list)  
  
message = "apples*2"  
key, value = message.split('*')  
  
print("Key:", key, "Value:", value)
```

Output

```
['apple', 'banana', 'orange']  
Key: apples Value: 2
```

`strip()`: Removes leading and trailing whitespace characters (such as spaces, tabs, and newline characters) from a string

```
message = "    Hello, World!    "  
message = message.strip()  
  
print("'" + message + "'")
```

Output

```
|Hello, World!|
```

`lstrip()`: Removes leading whitespace characters (such as spaces, tabs, and newline characters) from a string

```
message = "    Hello, World!    "  
message = message.lstrip()  
  
print("'" + stripped_string + "'")
```

Output

```
|Hello, World!    |
```

`rstrip()`: Removes trailing whitespace characters (such as spaces, tabs, and newline characters) from a string

```
message = "    Hello, World!    "  
message = message.rstrip()  
  
print("'" + message + "'")
```

Output

```
|    Hello, World!|
```

String Operations

Concatenation

To combine two strings the concatenation operator (+) can be used.

```
# Syntax  
string_operand + string_operand
```

When two strings are concatenated, a new string object is created which includes all the characters from the string to the left of the operator and all the characters of the string to the right of the operator.

```
greeting = "Hello "  
name = "John"  
  
print(greeting + name)  
  
question = "How are you today?"
```

```
print(greeting + name + ". " + question)
```

Output

```
Hello John  
Hello John. How are you today?
```

Note!

String concatenation operations are always evaluated left to right.

Repetition

The repetition operator (*) is used to concatenate multiple copies of a string.

```
# Syntax  
string * int
```

The left operand of the repetition operator must be a string and the right operand must be a whole number.

```
print("Hello" * 5)
```

Output

```
HelloHelloHelloHelloHello
```

Warning!

A string will contain no characters when the right operand is less than or equal to 0.

Escape Sequences

Escape sequences in Python are special characters that are used to represent certain non-printable or special characters within strings.

Below is a list of common escape sequences that can be used when printing in Python:

- "\": Backslash
- "'": Single quote
- "\"": Double quote
- "\n": New Line
- "\t": Tab
- "\v": Vertical tab

```
print("Hello\tWorld.\n\nHello \'World.\\\'\\nHello\\v\\v\\\"World.\\\"")
```

Output

```
Hello   World.  
  
Hello 'World.'  
Hello  
  
"World."
```

Indexing

Characters in a string can be accessed by using an index. To index a string, use a set of square brackets after the string reference. Within the square brackets indicate the character you want to access using an integer value.

```
# Syntax
string[index]
```

The first character in a string is index 0, with each additional character's index incrementing from there. The text below demonstrates the string "Hello John" and the corresponding indices of the characters underneath each character.

```
Hello John
0123456789
```

Examples

```
message = "Hello John"

print(message[0])
print(message[4])
print(message[8])
```

Output

```
H
o
h
```

You can also use a negative index value. This will allow you to index characters from the end of the string.

```
message = "Hello John"

print(message[-1])
print(message[-4])
print(message[-9])
```

Output

```
n
J
e
```

IndexError

An IndexError occurs when you try to reference a character that doesn't exist.

```
message = "Hello John"

print(message[99])
```

Output

```
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
IndexError: string index out of range
```

Immutable

Strings are an immutable type. This means that the characters of a string value can never be changed.

Alert!

Since strings are an immutable type, attempting to assign a value to an element of the string will cause a `TypeError`.

Example

```
message[0] = 'J'
```

Output

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

Introduction to Logic and Loops

In this lesson, you will learn about the basics of programming and why it's important.

Control structures are the building blocks of any programming language, allowing you to control the flow of your code. They determine the sequence in which statements are executed and can be categorized into three main types:

- Sequential: Statements are executed in the order they are written.
- Selection: Statements are executed based on a condition (e.g., if-else).
- Iteration: Statements are executed repeatedly (e.g., loops).

In Python, the primary control structures are conditional statements (`if`, `elif`, and `else`) and loops (`for` and `while`).

```
number_of_bananas_eaten = 10

# Code example of control structures
if number_of_bananas_eaten > 0:
    print("number is positive")
elif number_of_bananas_eaten == 0:
    print("number is zero")
else:
    print("number is negative")
```

Output

```
number is positive
```

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)

- [Escape Sequences](#)
- [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)
- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)
 - [Boolean Expressions](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Ternary Operator \(Conditional Operator\)](#)
 - [Membership Operators](#)
 - [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)
 - [Loop Control Statements](#)
 - [Infinite Loops](#)
 - [Nesting Loops](#)
 - [Common Loop Patterns](#)
 - [Choosing the "Right" Loop](#)
 - [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

Importance of Logic and Loops in Programming

Logic and loops are crucial components in programming as they enable the implementation of complex algorithms and efficient code execution.

They help:

- **Make decisions:** Conditional statements allow your program to make decisions based on specific conditions, leading to different outcomes.
- **Automate repetitive tasks:** Loops enable you to perform the same operation multiple times without writing the same code repeatedly, making your code more efficient and easier to maintain.
- **Implement complex algorithms:** Many algorithms rely on control structures, such as sorting and searching algorithms, which are essential in computer science and software development.

```
# Code example of loops
for count in range(10):
    print(count)
```

Output

```
0 1 2 3 4 5 6 7 8 9
```

Indentation

When writing code, it's important to follow certain conventions to make your code readable and maintainable. One such convention is proper indentation.

In Python, indentation is essential for defining code blocks within control structures, loops, and functions. Unlike other programming languages that use brackets or keywords to denote code blocks, Python relies on indentation.

Here are some key points:

- **Consistency:** Use the same number of spaces or a tab character for each level of indentation in your code. Most Python developers use 4 spaces.
- **Required:** Indentation is not optional in Python; it determines the scope of code blocks and directly impacts the code's execution.
- **Readability:** Proper indentation improves code readability and makes it easier to understand and maintain.

```
number_of_bananas_eaten = 5

# Example of indentation
if number_of_bananas_eaten > 0:
    print("number is positive")
if number_of_bananas_eaten % 2 == 0:
    print("number is even")
else:
    print("number is not positive")
```

Output

```
number is positive
number is not positive
```

In this example, the indents determine which print statements belong to which `if` or `else` block.

Conditional Statements

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)
 - [Escape Sequences](#)
 - [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)
- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)
 - [Boolean Expressions](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Ternary Operator \(Conditional Operator\)](#)
 - [Membership Operators](#)
 - [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)

- [Loop Control Statements](#)
- [Infinite Loops](#)
- [Nesting Loops](#)
- [Common Loop Patterns](#)
- [Choosing the "Right" Loop](#)
- [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

if, elif, and else Statements

Conditional statements allow your program to make decisions based on specific conditions. Python has three keywords for this purpose: `if`, `elif` (short for else if), and `else`.

- `if`: Evaluates a condition and executes the following code block if the condition is `True`.
- `elif`: Checks another condition if the previous `if` statement is `False`. You can have multiple `elif` statements.
- `else`: Executes the code block if none of the previous conditions are `True`.

```
# Define a variable representing the person's age
age = 18

# Use if-elif-else statements to determine the appropriate label for the person's age
if age < 13:
    print("Child")
elif age < 18:
    print("Teenager")
else:
    print("Adult")
```

Output

```
Adult
```

Nested Conditionals

Nested conditionals are conditional statements inside other conditional statements. They can be useful when you need to check multiple conditions or combinations of conditions.

```
# Define two variables representing horizontal and vertical positions
horizontal_position = 3
vertical_position = 5

# Use nested if-else statements to determine the quadrant of the position
if horizontal_position > 0:
    if vertical_position > 0:
        print("Both horizontal_position and vertical_position are positive")
    else:
        print("horizontal_position is positive, but vertical_position is not")
else:
    if vertical_position > 0:
        print("vertical_position is positive, but horizontal_position is not")
```

```
else:  
    print("Both horizontal_position and vertical_position are not positive")
```

Output

```
Both horizontal_position and vertical_position are positive
```

Boolean Expressions

A Boolean Expression is an expression of a programming language that results in a boolean value (True or False). A Boolean expression may be composed of a combination of the Boolean constants True or False, Boolean-typed variables, Boolean-valued operators, and Boolean-valued functions.

Most boolean expressions are created using binary operators. Binary operators require two operands to complete the expression, the syntax of which is:

```
operand operator operand
```

The operands are the values you are comparing. The operands can be values from literal values, variables, functions, or other expressions.

Comparison Operators

Comparison operators are used to compare two values.

Common comparison operators in Python are:

- ==: Equal to
- !=: Not equal to
- <: Less than
- >: Greater than
- <=: Less than or equal to
- >=: Greater than or equal to

```
age = 5  
friends_age = 10  
  
print(age == friends_age)  
print(age != friends_age)  
print(age < friends_age)  
print(age > friends_age)  
print(age <= friends_age)  
print(age >= friends_age)
```

Output

```
False  
True  
True  
False  
True  
False
```

Logical Operators

Logical operators are used to combine multiple conditions. Logical Operators can only have boolean expressions as their operands. The and operator and the or operator are binary operators. The not operator is a unary operator, meaning it only has one operand.

Python has three logical operators:

- and: Evaluates to True if both operands are True.
- or: Evaluates to True if at least one of the operands is True.
- not: Evaluates to True if the operand is False, and False if the operand is True.

```
# Define two variables representing ages
age = 5
friends_age = 10

# Check whether both ages are positive
if age > 0 and age2 > 0:
    print("Both age and friends_age are positive")

# Check whether at least one age is greater than 9
if age > 9 or friends_age > 9:
    print("At least one of age or friends_age is greater than 9")

# Check whether age1 is not greater than 10
if not age > 10:
    print("age is not greater than 10")
```

Output

```
Both age and friends_age are positive
At least one of age or friends_age is greater than 9
age is not greater than 10
```

Ternary Operator (Conditional Operator)

Ternary operator, also known as conditional operator, is the only operator with three operands. This operator is used specifically when deciding between two values. It can be a more concise than writing if-else statement.

The Ternary Operator has the following syntax:

```
# Syntax
value_if_true if boolean_expression else value_if_false
```

Example:

```
# Define a variable representing a number
number_of_donuts = 5

# Use a conditional expression to determine whether the number is even or odd
message = "Even" if number_of_donuts % 2 == 0 else "Odd"

# Print the resulting label for the number
print(message)
```

Output

```
Odd
```

In this example, the ternary expression checks if `number_of_donuts` is even and assigns "Even" to the variable `message` if `True`, otherwise it assigns "Odd".

Membership Operators

Membership checking allows you to determine if a value is a member of a sequence (such as a list, tuple, or string). You can use the `in` and `not in` operators to check for membership.

These binary operators have the following syntax:

```
# Syntax
element in collection
element not in collection
```

where "element" is the item you are looking for, and "collection" is a reference to the collection to search for the "element" in.

Examples:

```
# Check if an item is in a list using the "in" keyword
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)

# Check if an item is not in a list using the "not in" keywords
print("orange" not in fruits)

# Check if a substring is in a string using the "in" keyword
text = "Hello, world!"

print("world" in text)
```

Output

```
True
True
True
```

Membership checking can be combined with loops and conditionals to create more powerful and expressive code.

```
# Define a list of fruits
fruits = ["apple", "banana", "cherry"]

# Define a fruit to search for
searched_fruit = "banana"

# Check if the searched fruit is in the list using membership checking
if searched_fruit in fruits:
    print("Found", searched_fruit)
else:
    print(searched_fruit, "not found")

# Check if the searched fruit is in the list using a loop
for fruit in fruits:
    if fruit == searched_fruit:
        print("Found", searched_fruit)
        break

# Check if the searched fruit is in the list using list comprehension and membership checking
if any(fruit == searched_fruit for fruit in fruits):
    print("Found", searched_fruit)
else:
```

```

print(searched_fruit, "not found")

# Use list comprehension and membership checking to filter a list
numbers = [1, 3, 5, 7, 9]

even_numbers = [number for number in range(1, 11) if number not in numbers]

print(even_numbers)

```

Combining membership checking with loops and conditionals allows you to write more concise and efficient code, and it makes your programs easier to read and understand.

Short Circuiting

Short circuiting occurs when a logical expression is evaluated only as far as necessary to determine its value. In Python, the and and or operators use short-circuit evaluation.

```

# Check if a number is both positive and even
number = -2

if number > 0 and number % 2 == 0:
    # statements when true
else:
    # statements when false

```

In this example, the and operator short-circuits the evaluation, meaning that the second condition (number % 2 == 0) is not evaluated if the first condition (number > 0) is False. This can improve the performance of your code.

Warning!

Remember to prioritize readability and maintainability over performance optimizations, especially for small-scale projects or when performance differences are negligible. Always profile and measure the performance of your code before optimizing it.

Loops

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)
 - [Escape Sequences](#)
 - [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)
- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)
 - [Boolean Expressions](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Ternary Operator \(Conditional Operator\)](#)
 - [Membership Operators](#)
 - [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)
 - [Loop Control Statements](#)

- [Infinite Loops](#)
- [Nesting Loops](#)
- [Common Loop Patterns](#)
- [Choosing the "Right" Loop](#)
- [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

for Loop

For loops are used to iterate over a sequence (such as a list, tuple, dictionary, or string) and is used in programming to execute a block of code multiple times, based on some condition or sequence of data. In this case, the sequence of data we're using is a list of fruits, which is defined on the first line of the code as:

```
fruits = ["apple", "banana", "cherry"]
```

The for loop itself starts on the second line of the code, and has the following structure:

```
for fruit in fruits:  
    print(fruit)
```

The first line of the for loop specifies the variable that we will use to represent each element in the sequence as we iterate over it. In this case, we're using the variable name "fruit".

The second line of the for loop is indented, which means it is part of the block of code that will be executed for each element in the sequence. In this case, we're using the `print()` function to output each fruit to the console.

As the for loop executes, the variable `fruit` takes on the value of each element in the `fruits` list, one at a time, and the code within the loop (in this case, the `print()` statement) is executed for each element. When there are no more elements in the sequence, the loop terminates and the program continues with the next line of code after the loop.

To summarize, this for loop is iterating over the `fruits` list, and for each element in the list, it is printing that element to the console. The end result is that the program will output each fruit in the list, one at a time, in the order they appear in the list:

```
apple  
banana  
cherry
```

Here's the example altogether:

```
fruits = ["apple", "banana", "cherry"]  
  
# print each fruit in the list  
for fruit in fruits:  
    print(fruit)
```

Output

```
apple  
banana  
cherry
```

Iterating Iterable Objects

You can use for loops to iterate through various iterable objects, such as lists, dictionaries, tuples, and strings.

Lists and Tuples

```
numbers = [1, 2, 3, 4, 5]  
  
# print each number in the list multiplied by 2  
for number in numbers:  
    print(number * 2)
```

Output

```
2  
4  
6  
8  
10
```

Dictionaries

Similar to the previous example, this is also an example of a for loop in Python, but this time we're using a dictionary as the sequence of data to iterate over. A dictionary is a type of data structure that stores key-value pairs, and in this example, we're using a dictionary to store information about a student.

In the for loop, we're using the `items()` method to return a list of key-value pairs in the dictionary. The loop variable `key` is used to represent each key in the dictionary, while `value` is used to represent the corresponding value for that key.

```
# Define a dictionary with student information  
student = {"name": "John", "age": 20, "grade": "A"}  
  
# Iterate over the items in the dictionary and print each key-value pair  
for key, value in student.items():  
    print(key, value)
```

Output

```
name John  
age 20  
grade A
```

Strings

```
# Define a string  
text = "Python"  
  
# Iterate over each character in the string and print the uppercase version  
for character in text:  
    print(character.upper())
```

Output

```
P  
Y  
T  
H  
O  
N
```

range() Function

The `range()` function is commonly used with `for` loops to repeat a block of code a specific number of times. It generates a sequence of numbers from a start value (inclusive) to a stop value (exclusive) with an optional step value.

```
# Iterate over the range 0-4 (exclusive)
for count in range(5):
    print(count)

# Iterate over the range 2-5 (exclusive)
for count in range(2, 6):
    print(count)

# Iterate over the range 1-9 (exclusive), stepping by 2
for count in range(1, 10, 2):
    print(count)
```

enumerate() Function

The `enumerate()` function allows you to loop through an iterable with an index, which can be useful when you need to keep track of the current iteration.

```
# Define a list of fruits
fruits = ["apple", "banana", "cherry"]

# Iterate over each fruit in the list, along with its index, and print them
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output

```
0 apple
1 banana
2 cherry
```

List Comprehensions

List comprehensions are a concise way to create lists using a single line of code. You can include conditionals to create more complex lists.

In this example, we're using a list comprehension to create a new list called `squares`, which contains the squares of all the odd numbers in the original list `numbers`.

The list comprehension itself is defined in a single line of code and has the following structure:

```
squares = [number ** 2 for number in numbers if number % 2 == 1]
```

The first part of the list comprehension, `number ** 2`, defines the operation to perform on each element in the `numbers` list. In this case, we're squaring each number.

The second part of the list comprehension, `for number in numbers`, specifies how to iterate over the original list `numbers`. In this case, we're using a for loop to iterate over each element in `numbers`.

The third part of the list comprehension, `if number % 2 == 1`, is an optional conditional statement that filters the elements in `numbers` based on a certain condition. In this case, we're only including odd numbers in the new list `squares`.

As the list comprehension executes, it applies the `number ** 2` operation to each odd number in the `numbers` list, and adds the resulting value to the new `squares` list. When there are no more odd numbers in the `numbers` list, the list comprehension completes and the program moves on to the next line of code.

Here's the full example:

```
# Define a list of numbers
numbers = [1, 2, 3, 4, 5]

# Use a list comprehension to create a list of squares of odd numbers in the original list
squares = [number ** 2 for number in numbers if number % 2 == 1]

# Print the resulting list of squares
print(squares)
```

Output

```
[1, 9, 25]
```

while Loop

While loops are another type of loop used in programming to execute a block of code multiple times, based on a specific condition. The while loop will continue to execute the block of code until the condition is no longer `True`.

Here is an example of a while loop in Python:

```
# Initialize a counter variable to 1
count = 1

# Use a while loop to repeatedly print the value of the counter variable and increment it
while count <= 5:
    print(count)
    count += 1
```

Output

```
1, 2, 3, 4, 5
```

In this example, we're using a while loop to repeatedly print the value of a counter variable and increment it by 1, until the counter reaches 5.

The first line of the code initializes the `count` variable to 1.

The second line of the code starts the while loop, with the following structure:

```
while count <= 5:
```

This specifies the condition that must be `True` in order for the loop to continue executing. In this case, the loop will continue to execute as long as `count` is less than or equal to 5.

The third line of the code is indented, indicating that it is part of the block of code that will be executed repeatedly for each iteration of the loop. In this case, we're using the `print()` function to output the value of `count` to the console.

The fourth line of the code increments the value of `count` by 1, using the `+=` operator.

As the `while` loop executes, the program checks the condition `count <= 5`. If the condition is `True`, the block of code within the loop (in this case, the `print()` statement) is executed, and the value of `count` is incremented by 1. This process continues until the condition is no longer `True`, at which point the loop terminates and the program continues with the next line of code.

Loop Control Statements

Loop control statements change the flow of loops during execution.

- `break`: Terminates the loop and exits.
- `continue`: Skips the remaining code in the current iteration and starts the next iteration.

```
# Use a for loop to iterate over the range of integers from 1 to 9 (exclusive)
for number in range(1, 10):
    # Use the continue statement to skip over multiples of 3
    if number % 3 == 0:
        continue

    # Print all other integers
    print(number)
```

Output

```
1, 2, 4, 5, 7, 8
```

```
# Use a for loop to iterate over the range of integers from 1 to 9 (exclusive)
for number in range(1, 10):
    # Use the break statement to exit the loop if the integer is greater than 5
    if number > 5:
        break

    # Print all other integers
    print(number)
```

Output

```
1, 2, 3, 4, 5
```

else Statement

The `else` keyword can be used with a `while` loop to execute a block of code once the loop's condition becomes `False`.

```
# Initialize a counter variable to 1
count = 1

# Use a while loop to repeatedly print the value of the counter variable and increment it
# Add an else clause to execute a statement when the loop terminates normally
while count <= 5:
```

```
print(count)

count += 1
else:
    print("Loop finished")
```

Output

```
1, 2, 3, 4, 5
Loop finished
```

Infinite Loops

Infinite loops occur when the loop's condition never becomes `False`, causing the loop to run indefinitely. To avoid infinite loops, ensure that there's a way for the loop condition to become `False` at some point during execution.

Example of an infinite loop:

Warning!

Don't run this code, as it will result in an infinite loop.

```
# Initialize a counter variable to 1
count = 1

# Use a while loop to repeatedly print the value of the counter variable and increment it
# This loop will run indefinitely, since the condition (count > 0) is always True and there is no way to break out of the loop
while count > 0:
    print(count)

    count += 1
```

How to avoid the infinite loop:

```
# Initialize a counter variable to 1
count = 1

# Use a while loop to repeatedly print the value of the counter variable and increment it
# Add a conditional statement to break out of the loop when the counter reaches a value greater than 5
while count > 0:
    print(count)

    count += 1

    if count > 5:
        break
```

Output

```
1, 2, 3, 4, 5
```

Nesting Loops

Nested loops are loops within other loops. They can be useful for iterating through multidimensional data structures, such as lists of lists or grids.

Example:

```
# Define a matrix as a list of lists
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Use nested for loops to iterate over the matrix and print each element to the console
for row in matrix:
    for element in row:
        print(element, end=' ')

    # Use the print() function with no arguments to print a newline character after each row
    print()
```

Output

```
1 2 3
4 5 6
7 8 9
```

Common Loop Patterns

Loop Counters

Loop counters are variables used to count the number of iterations in a loop.

```
# Initialize a counter variable to 0
count = 0

# Use a for loop and an if statement to count the even numbers between 1 and 10
for number in range(1, 11):
    if number % 2 == 0:
        count += 1

# Use the print() function to display the count of even numbers
print("There are", count, "even numbers between 1 and 10")
```

Output

```
There are 5 even numbers between 1 and 10
```

Accumulators

Accumulators are variables used to accumulate a value during a loop's iterations, such as summing or multiplying numbers.

```
# Initialize a sum variable to 0
sum = 0

# Use a for loop and an accumulator pattern to compute the sum of the integers from 1 to 5
for number in range(1, 6):
    sum += number

# Use the print() function to display the sum of the integers
print("The sum of numbers from 1 to 5 is", sum)
```

Output

The sum of numbers from 1 to 5 is 15

Searching and Filtering

Loops can be used to search for specific elements or filter elements based on certain conditions.

Searching

```
# Define a list of numbers and a target value
numbers = [1, 5, 8, 12, 15, 21]
target = 12

# Use a for loop and an if statement to search for the target value in the list
for number in numbers:
    if number == target:
        print("Found", target)

        break
```

Output

Found 12

Filtering

```
# Define a list of numbers
numbers = [1, 5, 8, 12, 15, 21]

# Use a list comprehension to filter the even numbers from the list
filtered_numbers = [number for number in numbers if number % 2 == 0]

# Use the print() function to display the filtered even numbers
print("Even numbers:", filtered_numbers)
```

Output

[8, 12]

Choosing the "Right" Loop

When you have a choice between a for loop and a while loop, choose the one that best fits your use case:

Use a for loop when you want to iterate over a sequence (e.g., list, tuple, string) or when you know the exact number of iterations.

```
# Choose the appropriate loop structure for the task at hand to make the code more efficient

# Good example:
# Using a 'for' loop to iterate over a range of numbers
for num in range(5):
    print(num)

# Bad example:
# Using a 'for' loop to wait for user input until a certain condition is met
condition_met = False
```

```

for i in range(10):
    user_input = input("Enter a value: ")

    if user_input == "stop":
        condition_met = True

        break

if condition_met:
    print("Condition met!")
else:
    print("Condition not met.")

```

Use a while loop when the number of iterations depends on a condition that may change during the loop's execution.

```

# Choose the appropriate loop structure for the task at hand to make the code more efficient

# Good example:
# Using a 'while' loop to wait for user input until a certain condition is met
condition_met = False

while not condition_met:
    user_input = input("Enter a value: ")

    if user_input == "stop":
        condition_met = True

# Bad example:
# Using a 'while' loop to iterate over a fixed sequence of values
values = [1, 2, 3, 4, 5]
index = 0

while index < len(values):
    print(values[index])

    index += 1

```

Best Practices

Add comments to explain your code. Use consistent indentation and spacing and follow the PEP 8 style guide for Python code.

```

# Use consistent and appropriate indentation to make the code more readable and maintainable
# Bad example:
# Using two tabs for indentation
for number in range(10):
    print(number)

# No indentation
for number in range(10):
print(number)

# Good example:
# Using single indentation for each level of the code block
for number in range(10):
    print(number)

```

Try to avoid unnecessary computations, especially in loops. For example, if you need to calculate a value that doesn't change in a loop, move the calculation outside the loop to avoid repeated computations.

```

# Inefficient code: This code has a nested loop and performs the same
# multiplication operation twice in every iteration, resulting in an
# unnecessarily long execution time.

```

```

result = []
for first_number in range(1000):
    for second_number in range(1000):
        result.append(first_number * 2 + second_number * 2)

# Efficient code: This code has eliminated the redundant multiplication
# by storing the result of each multiplication in a separate variable
# before appending it to the result list.
result = []
for first_number in range(1000):
    first_number_doubled = first_number * 2
    for second_number in range(1000):
        second_number_doubled = second_number * 2
        result.append(first_number_doubled + second_number_doubled)

```

Importing Modules

Python has many built-in functions that are available for you to use without doing any special to access them. These functions allow you to perform the most basic of functionality in Python.

Python also has a vast library of modules that you can *import* in order to perform more sophisticated functionality.

You've already learned that a module is a Python .py file that consists of Python source code. Modules can define functions, classes, and variables. In order to use the entities defined within another module, the module you are developing needs to import that module (or part of the module).

import Statement

To import a module from the Python Standard Library, use the `import` keyword followed by the module's name.

The following code snippet imports the `math` module, defined within the Python Standard Library. The `math` module contains functions to do more advanced mathematical operations.

```

# Import the math module, which provides mathematical functions and
# constants.
import math

# Invoke the sqrt() function from the math module to compute the square
# root of a number.
print(math.sqrt(16))

```

Output

```
4.0
```

from ... import Statement

To import a specific element from a module, you can use the `from [module] import [function]` statement.

The follow code snippet imports the `sqrt` function from the `math` module. Take not that in the example above, the `sqrt` function call was preceded by the name of the module it's defined in. When you import using `from ... import`, the module is not used to invoke the function.

```

# Import the sqrt() function from the math module.
from math import sqrt

# Invoke the sqrt() function to compute the square root of a number.
print(sqrt(16))

```

Random Module

random() Function

The random module provides various functions to generate random numbers. Some common functions include:

- `random.random()`: Returns a random float between 0 and 1.
- `random.randint(a, b)`: Returns a random integer between a and b (inclusive).
- `random.choice(sequence)`: Returns a random element from a sequence (such as a list, tuple, or string).
- `random.shuffle(sequence)`: Shuffles the elements of a sequence in place.

```
# Import the random module, which provides functions for generating
# random numbers and values
import random

# Use the random() function to generate a random float between 0 and 1
print(random.random())

# Use the randint() function to generate a random integer between two
# values (inclusive)
print(random.randint(1, 6))

# Define a list of fruits
fruits = ["apple", "banana", "cherry"]

# Use the choice() function to randomly select an element from a list
print(random.choice(fruits))

# Use the shuffle() function to randomly reorder the elements of a
# list
random.shuffle(fruits)
print(fruits)
```

Output

```
0.5873688413439251
3
banana
["cherry", "apple", "banana"]
```

Remember that these examples are for illustration purposes only. The actual output will be different each time you run the code since the random module generates random values.

File Input/Output (File I/O)

Contents

- [Working With Strings](#)
 - [String Methods](#)
 - [String Operations](#)
 - [Escape Sequences](#)
 - [Indexing](#)
- [Introduction to Logic and Loops](#)
 - [Importance of Logic and Loops in Programming](#)
 - [Indentation](#)

- [Conditional Statements](#)
 - [if, elif, and else Statements](#)
 - [Nested Conditionals](#)
 - [Boolean Expressions](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Ternary Operator \(Conditional Operator\)](#)
 - [Membership Operators](#)
 - [Short Circuiting](#)
- [Loops](#)
 - [for Loop](#)
 - [List Comprehensions](#)
 - [while Loop](#)
 - [Loop Control Statements](#)
 - [Infinite Loops](#)
 - [Nesting Loops](#)
 - [Common Loop Patterns](#)
 - [Choosing the "Right" Loop](#)
 - [Best Practices](#)
- [Importing Modules](#)
 - [import Statement](#)
 - [from ... import Statement](#)
- [Random Module](#)
 - [random\(\) Function](#)
- [File Input/Output \(File I/O\)](#)
 - [Opening and Closing Files](#)
 - [Reading and Writing Files](#)
 - [with Statements](#)
 - [DictReader and DictWriter](#)

Opening and Closing Files

To work with files in Python, you need to open them first. The `open()` function is used to open a file and returns a file object. It takes two arguments: the file's path and an optional mode.

Common modes include:

- 'r': Read mode (default)
- 'w': Write mode (overwrites the file if it exists, creates a new one otherwise)
- 'a': Append mode (appends data to the file if it exists, creates a new one otherwise)
- 'x': Exclusive creation mode (creates a new file, raises an error if the file exists)
- 'b': Binary mode (used for non-text files, such as images)

To close a file, use the `close()` method.

```
# Use the open() function to open a file in read mode
file = open("example.txt", "r")

# Perform file operations here, such as reading or processing
# the contents of the file

# Use the close() method to close the file when done with file
# operations
file.close()
```

Reading and Writing Files

To read or write data from/to a file, use the following methods:

- `read()`: Reads the entire content of the file as a string.

- `readline()`: Reads a single line from the file.
- `readlines()`: Reads all lines from the file and returns them as a list of strings.
- `write(string)`: Writes the specified string to the file.

Reading Example

```
# Reading a file  
# Use the open() function to open a file in read mode  
file = open("example.txt", "r")  
  
# Use the read() method to read the contents of the file into a string  
# variable  
content = file.read()  
  
# Use the print() function to display the contents of the file to the  
# console  
print(content)  
  
# Use the close() method to close the file when done reading its  
# contents  
file.close()
```

Writing Example

```
# Writing to a file  
# Use the open() function to open a file in write mode  
file = open("output.txt", "w")  
  
# Use the write() method to write a string to the file  
file.write("Hello, world!")  
  
# Use the close() method to close the file when done writing to it  
file.close()
```

with Statements

The `with` statement is a more convenient and safer way to work with files, as it automatically closes the file when the block of code is exited.

```
# Use the with statement to open a file in read mode  
# The file will be automatically closed after the block of code is  
# executed  
with open("example.txt", "r") as file:  
    # Use the read() method to read the contents of the file into a  
    # string variable  
    content = file.read()  
  
    # Use the print() function to display the contents of the file  
    # to the console  
    print(content)
```

Using the `with` statement is considered best practice for file handling, as it helps prevent issues like file leaks or accidentally leaving a file open.

DictReader and DictWriter

Reading and writing to files is a task that is often done in conjunction with looping. The `csv` module is encapsulated with functionality that may be used to read data from a comma-delimited file into a dictionary, namely the `DictReader` method. Similarly, the `csv` module contains functionality that may be used to write dictionary data to a file. Namely the `DictWriter` method. To use the `csv` module, it must be imported:

```
import csv

#OR

from csv import DictReader
from csv import DictWriter
```

DictReader

The DictReader function is used to read CSV files. It allows you to read the contents of a CSV file and treat each row as a dictionary, where the keys are the column names and the values are the corresponding values in that row. Consider the following comma-delimited file called `fruit_quantity.csv`:

```
Item,Quantity
Apples,26
Oranges,12
Bananas,52
Pears,12
Plums,32
```

Using the open context manager, the file is opened and the DictReader function is used to create a DictReader object. By passing the file handle to the DictReader function, the reader 'object' is populated with the contents of the comma-delimited file.

The reader object is an iterator which allows you to iterate through the rows of the csv file, with each row represented as a dictionary. Since the reader object is an iterator, a loop may be used to iterate through its contents:

```
with open('fruit_quantity.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row['Item'], row['Quantity'])
```

Output:

```
Apples 26
Oranges 12
Bananas 52
Pears 12
Plums 32
```

DictWriter

The DictWriter function is used to write data into CSV files. It allows you to write rows of data to a CSV file based on dictionaries, where the keys are the column names and the values are the corresponding values for each column. Consider the following dictionary containing Names and Ages of 4 people:

```
data = {
    'Annie': 43,
    'Brenda': 36,
    'Chris': 19,
    'David': 29
}
```

When writing to a csv file, it is important to include a header row which contains the names of the column headers.

Note!

The column headers are not included in the dictionary shown above, however we know that each key-value pair contains a 'Name' and an 'Age', so these would be appropriate column headers.

In the code below:

- The `field_names` list specifies the column names you want to use in the CSV file.
- The `with` statement is used to open the file named `output.csv` in write mode.
- A `DictWriter` object named `writer` is created with the file handle and the specified field names.
- The `writeheader()` method of the `DictWriter` object is called to write the header row with the column names to the CSV file.
- The data from the dictionary is written to the CSV file using the `writerow()` method of the `DictWriter` object in a loop, where each iteration corresponds to a row in the CSV file. The dictionary passed to `writer.writerow()` contains the column names as keys and the corresponding values from the data dictionary.

```
import csv

data = {
    'Annie': 43,
    'Brenda': 36,
    'Chris': 19,
    'David': 29
}

field_names = ['Name', 'Age']

with open('output.csv', 'w', newline='') as file:
    writer = csv.DictWriter(file, fieldnames=field_names)

    # Write the header row with column names
    writer.writeheader()

    for name, age in data.items():
        writer.writerow({'Name': name, 'Age': age})
```

A `.csv` file called `output.csv` is created. The file contents are as follows:

```
Name, Age
Annie, 43
Brenda, 36
Chris, 19
David, 29
```

 Red River College logo

Search



From here you can search these documents. Enter your search terms below.